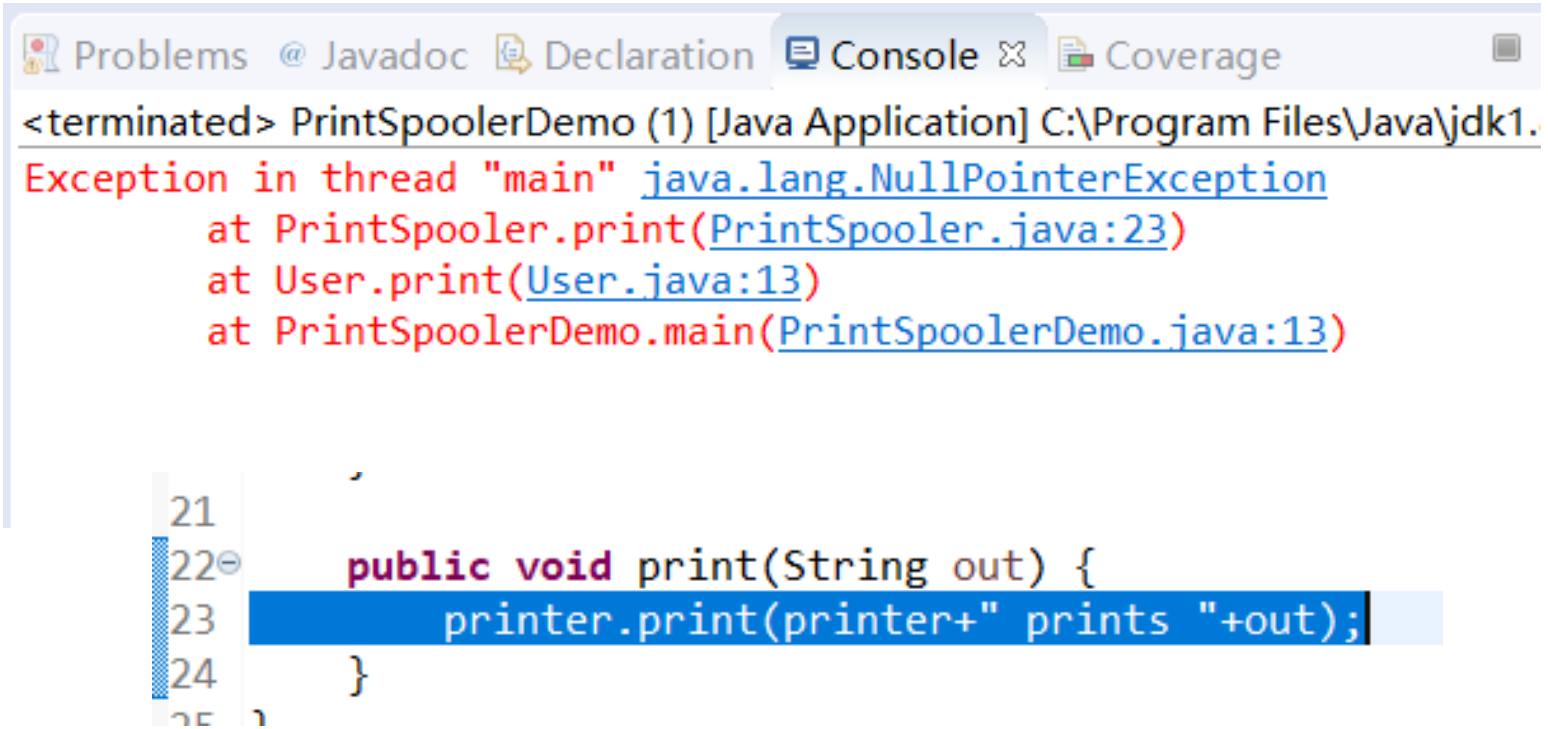
Part 7 Exception Handling

Software College of NEU



A Common Exception

- **java.lang.NullPointerException**



Contents



- What is an Exception
- Generating an Exception (throw/throws)
- Dealing with Exceptions (try/catch/finally)
- Checked vs Unchecked Exceptions
- How to write your own Exception

1. What is an Exception



Common Programmer faults



- Incorrect implementation of program specification
- Variables that have not been assigned an object
- Variables with incorrectly assigned values
- Index out of bound
- Inconsistent or inappropriate object state
- Path or file not found
- Infinite loops
- Divide by zero errors…. & …….. the list is endless

Programmers may get it right, but faults still occur!

Common Programmer faults

- Incorrect implementation of program specification
- Divide by zero errors…. & …….. the list is endless

- **Abnormal events often arise from the environment:**
  - **Incorrect URL entered**
  - **Network interruption**
- **File processing is particularly error-prone:**
  - **Missing files**
  - **Lack of appropriate permissions**

Programmers may get it right, but faults still occur!

What should we do to provide strong applications?

- Anticipating Errors
  - Where are errors likely to occur
  - What is the nature of those errors
- Error handling
  - Can we detect the error
  - What should we do if the error occurs
- Error reporting
  - Who should we tell, what should we say, and how should we log the error?

What can we do until now?



- Ignore
  - Possible program crash, corrupted state, corrupted data
- Write a message to the screen & continue on
  - Not practical sometimes, who reads the message still have a corrupted state
- Block the request that caused the problem, write error to log and shut down program
  - Would this be satisfactory for a software product?

Exceptions Provide a structured approach to deal with abnormal events that occur in a program.

Exceptions



- When a code block encounters an abnormal condition (an exception condition) that it can't handle itself...
  - An Exception is created
  - The exception is thrown
  - Normal program flow-of-control stops
  - The program searches for code that will catch the exception (deal with the exception)
    - Abandons each method on the stack until a handler is found
  - After the exception is dealt with normal program flow starts again

Definition of Exception



- An exception is an abnormal event that occurs during the execution of a program that disrupts the normal flow of instructions.
- Throwing an exception:
  - When an abnormal event occurs the code creates an exception object and hands it off to the runtime system.
  - The exception object contains information about the exception, including its type and the state of the program when the error occurred.
- Catching an exception:
  - It happens when the exception object bubbles up through the call stack until an appropriate exception handler is found. The handler catches the exception.

Javadoc for java.lang.NullPointerException

```
public class NullPointerException extends
                                    RuntimeException
```

- **NullPointerException objects may be constructed by the virtual machine as if suppression were disabled and/or the stack trace was not writable.**

Constructors

Constructor and Description
`NullPointerException()` Constructs a `NullPointerException` with no detail message.
`NullPointerException(String s)` Constructs a `NullPointerException` with the specified detail message.

https://docs.oracle.com/javase/7/docs/api/java/lang/NullPointerException.html

Javadoc for java.lang.NullPointerException



- **Thrown when an application attempts to use null in a case where an object is required. These include:**
  - **Calling the instance method of a null object.**
  - **Accessing or modifying the field of a null object.**
  - **Taking the length of null as if it were an array.**
  - **Accessing or modifying the slots of null as if it were an array.**
  - **Throwing null as if it were a Throwable value.**
- **Applications should throw instances of this class to indicate other illegal uses of the null object.**

https://docs.oracle.com/javase/7/docs/api/java/lang/NullPointerException.html

Javadoc for java.lang.NullPointerException



java.lang

Class NullPointerException

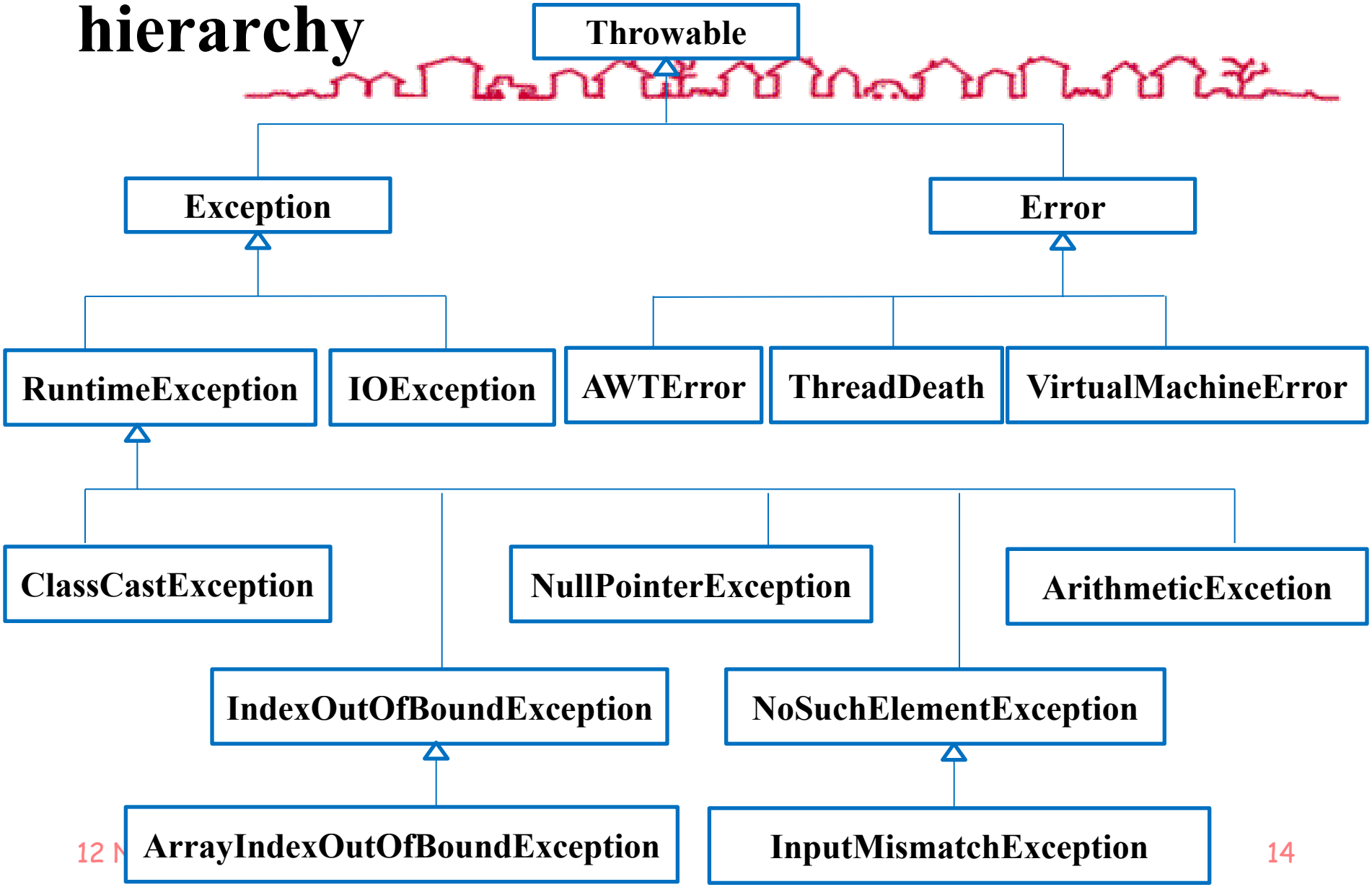
java.lang.Object
  java.lang.Throwable
    java.lang.Exception
      java.lang.RuntimeException
        java.lang.NullPointerException

All Implemented Interfaces:

Serializable

https://docs.oracle.com/javase/7/docs/api/java/lang/NullPointerException.html

A portion of class Throwable inheritance hierarchy



2. Generating an Exception

throw/throws



Throwing an exception



- `Throwable` serves as the base class for an entire family of exception classes, declared in `java.lang` package, that your program can instantiate and throw.
- To throw an exception, you must simply,
  - Construct an exception object
  - Then, use the keyword `throw` and throw the object

```
throw new SomeThrowableExceptionType();
```

- You can't throw just any object as an exception, however—only those objects whose classes descend from Throwable.

Throwing an exception

```
public class BankAccount
{
   public void withdraw(double amount)
   {
      if (amount > balance)
      {
         IllegalArgumentException exception
               = new IllegalArgumentException("Amount
                  exceeds balance");
         throw exception;
      }
      balance = balance - amount;
   }
   .
}
```

Problems @ Javadoc Declaration Console Coverage

```
<terminated> BankAccount [Java Application] C:\Program Files\Java\jdk1.8.0\bin\javaw.exe (2019年6月15日 上
Exception in thread "main" java.lang.IllegalArgumentException: Amount exceeds balance.
        at BankAccount.withdraw(BankAccount.java:19)
        at BankAccount.main(BankAccount.java:31)
```

Throwing an exception



- No need to store exception object in a variable:

```
public class BankAccount
{
   public void withdraw(double amount)
   {
      if (amount > balance)
      {
         throw new IllegalArgumentException(
                  "Amount exceeds balance");
      }
      balance = balance - amount;
   }
   . . .
}
```

Exception Specification



- `throws` keyword is used when a method contains code that may throw an exception. Then the throws clause can be used to declare it.
- It is also used if a method does not handle a checked exception. Then the method must throw it using the throws keyword.
- The throws keyword appears at the end of a method header.

```
public void withdraw(double amount)
        throws IllegalArgumentException
```

Exception Specification



- Throw an exception and transfer control to a handler for this exception type.

```
public class BankAccount
{
   public void withdraw(double amount)
                        throws IllegalArgumentException
   {
      if (amount > balance)
      {
         throw new IllegalArgumentException("Amount
                  exceeds balance");
      }
      balance = balance - amount;
   }
   . . .
}
```

Exception Specification

- A method can throw more than one exception:

```
public class className
{
    public void withdraw(double amount) throws
                    IOException, NullPointerException
    {
        // Method implementation
    }
    //Remainder of class definition
}
```

3. Dealing with Exceptions



try/catch/finally



The `try/catch` block

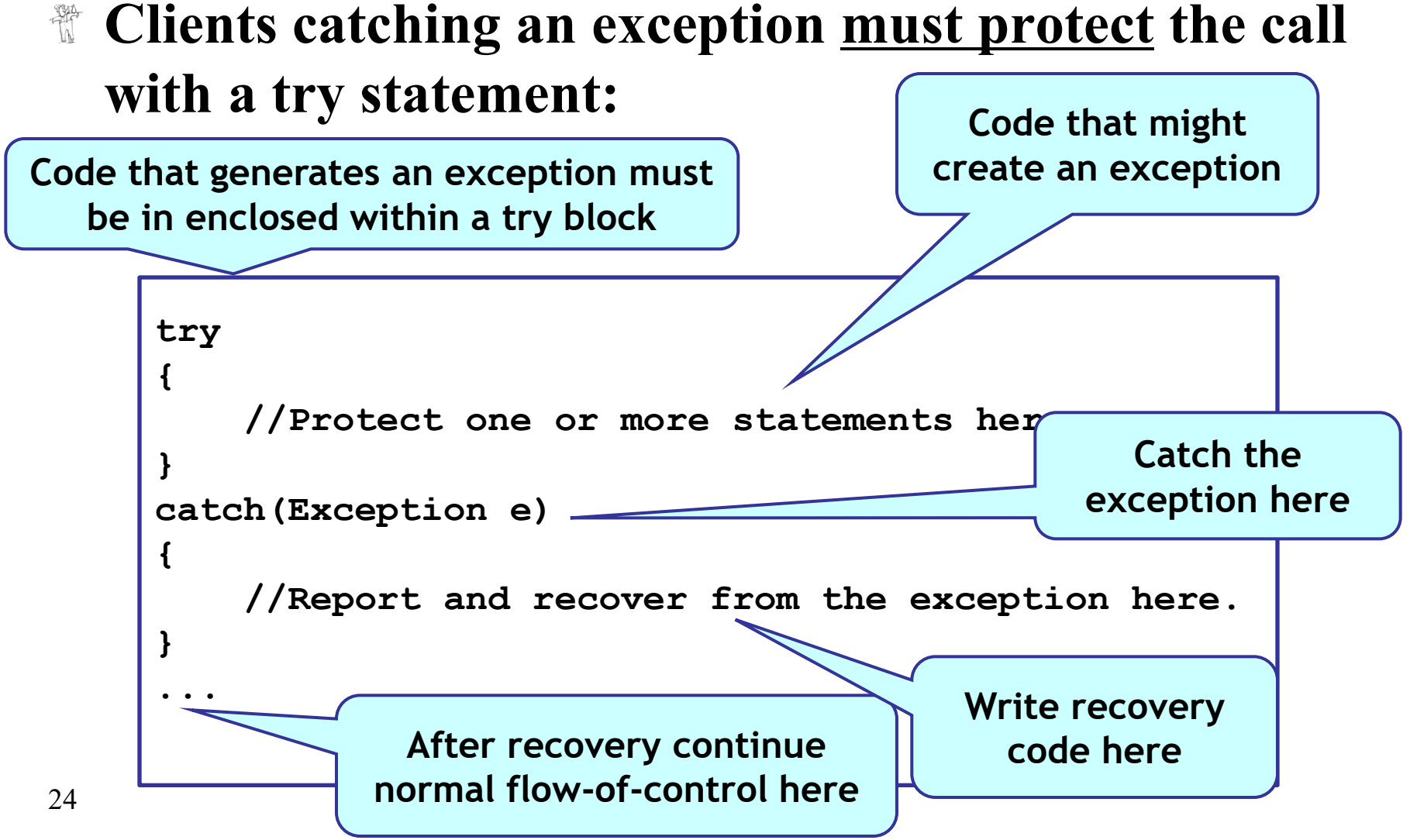


- To catch an exception in java ,we have to write a **try block with one or more catch clauses**.
  - The **try block** places a fence around a bit of code that is under the watchful eye of the associated catchers.
  - Each **catch clause** specifies one exception type that it is prepared to handle.

```
try
{
        //Java Statements that might create an exception
}
catch (ExceptionType variablename)
{
        //Java Statements to handle exception
}
```

The `try/catch` block

- **Clients catching an exception must protect the call with a try statement:**



The `try/catch` block



- If a code delimited by a try block throws an exception
  - The associated catch blocks of that try block will be examined by the JVM
  - If the JVM finds a catch clause that is prepared to handle the thrown exception, the program continues execution starting with the first statement of that catch clause.

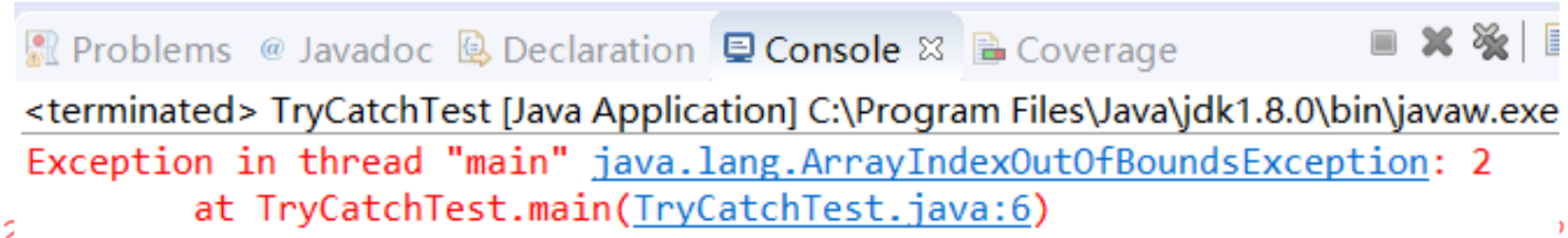
Example: Catching an exception

Observe the following code…

```
public class ExcepTest{

   public static void main(String args[]){
       int a[] = new int[2];
       System.out.println("Access element three :" + a[2]);
   }
}
```

The above is an array declared with 2 elements. Then the code tries to access the 3rd element of the array which throws an exception.



Example: Catching an exception

Solution:

```
public class ExcepTest{

   public static void main(String args[]){
      try{
         int a[] = new int[2];
         System.out.println("Access element three :" + a[2]);
      }
      catch(ArrayIndexOutOfBoundsException e){
         System.out.println("Exception thrown  :" + e);
      }
      System.out.println("Out of the block");
   }
}
```

Problems @ Javadoc Declaration Console Coverage

<terminated> TryCatchTest [Java Application] C:\Program Files\Java\jdk1.8.0\bin\

```
Exception thrown  :java.lang.ArrayIndexOutOfBoundsException: 2
Out of the block
```

Multiple catch blocks

- A try block can be followed by multiple catch blocks. The syntax for multiple catch blocks looks like the following:

```
try
{
    //Protected code
}
catch(ExceptionType1 e1)
{
    //Catch block 1
}
catch(ExceptionType2 e2)
{
    //Catch block 2
}
```

If an exception occurs and it matches one of the catch clauses, execute the first one that matches.

If no exception occurs, or an exception is thrown that doesn't match any catch clause, then skip the catch clauses.

Example: Catching multiple exceptions

```
try
{
   String filename = . . .;
   FileReader reader = new FileReader(filename);
   Scanner in = new Scanner(reader);
   String input = in.next();
   int value = Integer.parseInt(input);
}
catch (IOException exception)
{
   exception.printStackTrace();
}
catch (NumberFormatException exception)
{
   System.out.println("Input was not a number");
}
```

Catching Exceptions



- `catch (IOException exception) block`
  - `exception` contains reference to the exception object that was thrown
  - `catch` clause can analyze object to find out more details
  - `exception.printStackTrace()` : printout of chain of method calls that lead to exception

The `finally` clause



- The **`finally`** keyword is used to create a block of code that follows a try block.

```
try
{
     //Protect one or more statements here.
}
catch(Exception e)
{
     //Report and recover from the exception here.
}
finally {
     //Perform any actions here whether or
     //not an exception is thrown.
}
```

The `finally` clause



- The finally keyword is used to create a block of code that follows a try block.
- A finally block of code always executes, whether or not an exception has occurred.
- Using a finally block allows you to run any cleanup (housekeeping) type of statements that you want to execute, no matter what happens in the protected code.

The `finally` clause



- Exception terminates current method, which is danger as it may skip over essential code.
- Example

```
reader = new FileReader(filename);
Scanner in = new Scanner(reader);
readData(in);
reader.close(); // May never get here
```

  - Must execute `reader.close()` even if exception happens

- Use `finally` clause for code that must be executed "no matter what".

The `finally` clause

```
FileReader reader = new FileReader(filename);
try
{
   Scanner in = new Scanner(reader);
   readData(in);
}
finally
{
   reader.close(); // if an exception occurs, finally
                   // clause is also executed before
                   // exception is passed to its handler
}
```

The `finally` clause



- Executed when `try` block is exited in any of three ways:
  - After last statement of `try` block
  - After last statement of `catch` clause, if this `try` block caught an exception
  - When an exception was thrown in `try` block and not caught
- **Recommendation**: don't mix `catch` and `finally` clauses in same `try` block

4. Checked vs UnChecked Exceptions



Exception categories



- Unchecked exceptions (不受检查的异常)
  - All exceptions under `RuntimeException` and `Error` classes
  - Use for failures that is not checked by the compiler.
  - Where recovery is difficult.
- Checked exceptions (检查异常)
  - All other Exceptions under the `Throwable` class
  - Use for anticipated failures.
  - Where recovery is possible in most cases.

Checked Exceptions



- **Checked exceptions are checked at compile-time**
- **The compiler ensures that their use is tightly controlled.**
  - **A code block throwing a checked exception, should handle the exception using try-catch block or declare it using the throws exception clause. If not the code block will give a compile-time error.**
- **If used properly, failures may be recoverable.**

Checked Exception Example – Text input from file

```
try {
    BufferedReader reader =
        new BufferedReader(new FileReader("filename"));
    String line = reader.readLine();
    while(line != null) {
        do something with line
        line = reader.readLine();
    }
    reader.close();
}
catch(FileNotFoundException e) {
    the specified file could not be found
}
catch(IOException e) {
    something went wrong with reading or closing
}
```

Unchecked Exceptions



- These exceptions are not checked by the compiler.
- Cause program termination if not caught.
  - We need not catch Unchecked Exceptions if we expect the result of such a exception to be program termination
  - However, if there is a need to catch an Unchecked Exception, then an exception handler can be written for it, just like a Checked Exception
- `IllegalArgumentException` is a typical example.

Unchecked Exception Example

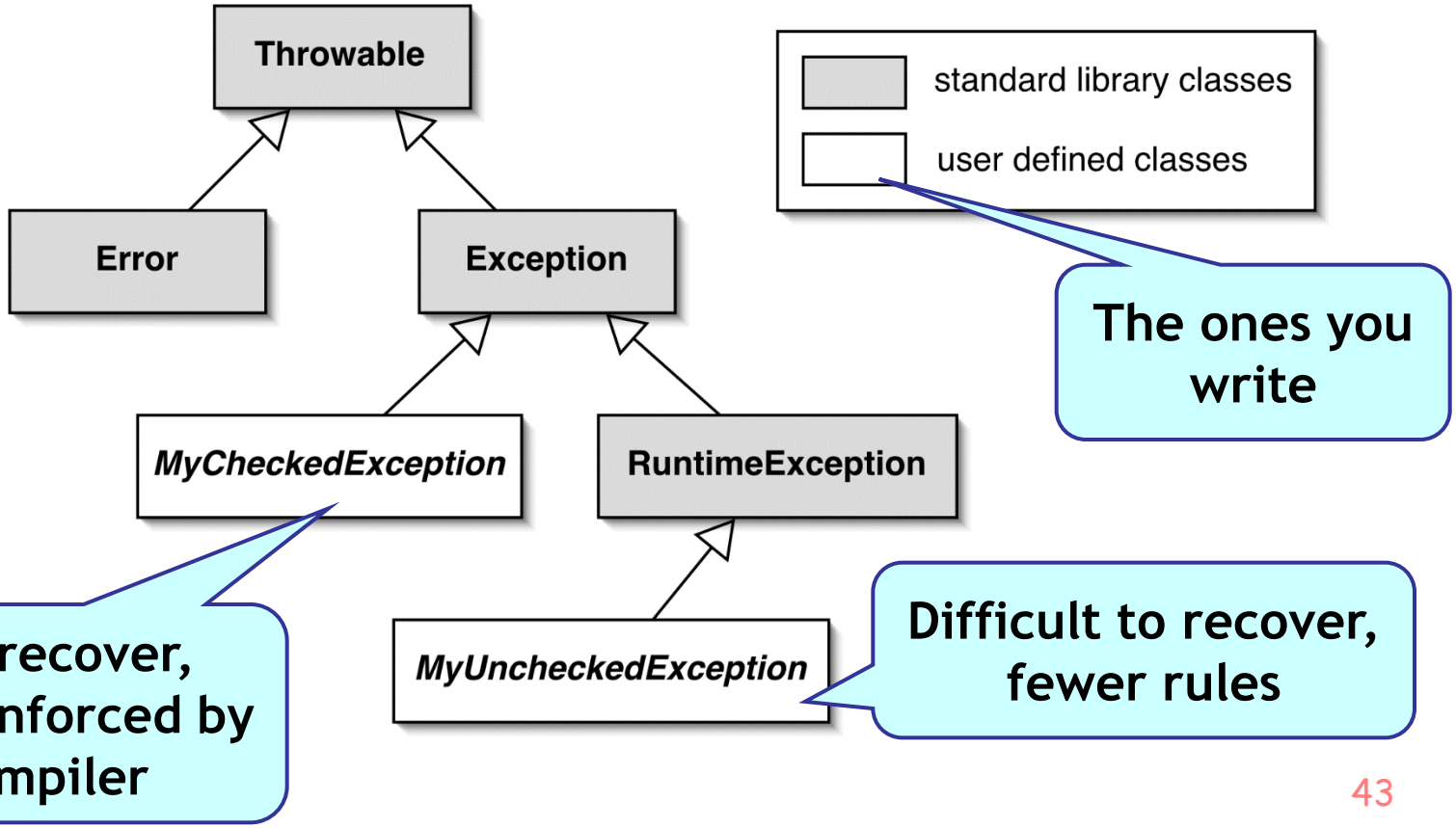
```
/**
 * Look up a name or phone number and return the
 * corresponding contact details.
 * @param key The name or number to be looked up.
 * @return The details corresponding to the key,
 *         or null if there are none matching.
 * @throws IllegalArgumentException if the key is null or empty
*/
public ContactDetails getDetails(String key)
{
    if(key == null) {
        throw new IllegalArgumentException("null key in getDetails");
    }
    if(key.trim().length() == 0) {
        throw new IllegalArgumentException("Empty key passed to
                                                getDetails");
    return book.get(key);
}
```

5. How to write your own Exceptios



Designing Your Own Execution Types

- You can design your own exception types– subclasses of `Exception` or `RuntimeException`



Designing Your Own Execution Types

- **Keep the following points in mind when writing your own exception classes:**
  - **All exceptions must be a child of Throwable.**
  - **If you want to write a checked exception that is automatically enforced by the Handle or Declare Rule, you need to extend the Exception class.**
  - **If you want to write an unchecked exception, you need to extend the RuntimeException class.**

```
public class InsufficientFundsException
                                    extends RuntimeException{
   public InsufficientFundsException() {}

   public InsufficientFundsException(String message){
      super(message);
   }
}
```

```
public class BankAccount {
   public void withdraw(double amount){
      if (amount > balance){
          throw new InsufficientFundsException(
                      "withdrawal of " + amount
                    + " exceeds balance of " + balance);
       }
      balance = balance - amount;
   }
   . . .
}
```

Homework



- **Add necessary exception handling program to your Gourmet coffee or library system.**

Part 7 Exception Handling

Software College of NEU

